

Technical Case Study: Autonomous Continuous Software Evolution and Sandboxed Security via Project Aegis-Evo

Summary

In modern artificial intelligence engineering, deploying autonomous agents to evolve large-scale, multi-file codebases remains an open challenge. While contemporary large language models (LLMs) achieve high success rates when patching isolated, single-file bugs, their efficiency degrades significantly when tasked with continuous, repository-scale software evolution across multiple interconnected subsystems. This capability barrier is compounded by severe runtime security concerns. Granting an autonomous agent the authority to write, compile, and execute code dynamically introduces a highly critical attack surface. If the agent ingests external code, untrusted third-party libraries, or maliciously crafted issue descriptions, it remains vulnerable to cross-domain Indirect Prompt Injection (XPIA). Such attacks can hijack the agent's decision loop, leading to arbitrary system command execution, host credential exfiltration, or container escape.

Project Aegis-Evo is designed as a research-driven prototype and secure systems-integration architectural blueprint rather than a finalized, production-scale commercial product. The architecture integrates a dynamic, information-flow-gated middleware with an Abstract Syntax Tree (AST)-guided dependency mapper and an ephemeral, hardware-isolated virtual runtime sandbox. This case study provides a technical evaluation of the Aegis-Evo prototype, detailing its architectural trust boundaries, structural code navigation engines, and sandboxed runtimes. It analyzes both the security efficacy and the performance overhead of the system, presenting a realistic, systems-level approach to securing autonomous software engineering pipelines.

Context and Constraints

Deploying autonomous developer agents (such as automated CI/CD patchers or pull-request reviewers) introduces a unique threat model that traditional application security (AppSec) tooling cannot address. Static security scanners and container posture management systems operate out-of-band and pre-deployment. They rely on identifying known vulnerability signatures (CVEs) or checking configurations against static policies. However, autonomous agents generate code dynamically and invoke external systems tools at runtime, creating emergent execution behaviors. Because the agent processes both natural language instructions (data) and executable code within the same context window, it is fundamentally vulnerable to cross-domain execution hijacking. A malicious actor can embed obfuscated commands in code comments, database records, or markup files. When parsed by the agent, these instructions bypass semantic system prompts and trigger unauthorized tool usage. To operate safely within enterprise development environments, Project Aegis-Evo was

designed under strict operational and security constraints:

- 1. **Host Kernel Isolation:** Legitimate but untrusted compilation steps and unit-test executions must run in environments that prevent direct access to the host operating system kernel, neutralizing container breakout vectors.
- 2. **Zero-Trust Network Egress:** Outbound network traffic from the execution environment must be locked down, restricting communication to allowlisted package registries (e.g., PyPI) to prevent dynamic data exfiltration.
- 3. **Information Flow Gating:** Low-integrity data ingested from untrusted external repositories must be restricted from influencing high-integrity system calls, filesystem writes, or command executions, independent of the planner's semantic state.
- 4. **Structural Dependency Mapping:** Code navigation tools must operate on structural, syntax-aware graphs rather than naive string or vector searches, minimizing the risk of "code tangling" where cycles of dependency overwhelm the agent's planning phase.

Problem Statement: The Codebase-Scale Evolution Barrier

Autonomous software engineering performance degrades significantly when transitioning from isolated, single-file bug repairs to continuous, multi-file software evolution. This degradation is driven by two primary challenges:

Structural Complexity and Code Tangling

In large-scale codebases, import paths and system dependencies are highly interconnected, creating overlapping dependency cycles. This structural complexity—frequently described as "code tangling"—makes localized modifications highly risky. A minor change in a low-level module can trigger transitively cascading logic failures and compilation breakages across unrelated upstream components. Naive semantic search engines (such as standard RAG or vector indexing) fail to capture these precise structural relations, leading the agent's planner to make incorrect assumptions, exhaust its context window, and ultimately fail the task.

Dynamic Execution Hijacking and the Escape Kill Chain

To evolve code successfully, an agent must compile modified files and execute verification suites. If the agent ingests a codebase containing an indirect prompt injection, it can be manipulated into executing arbitrary code. This vulnerability is structured as a five-stage system breakout kill chain:

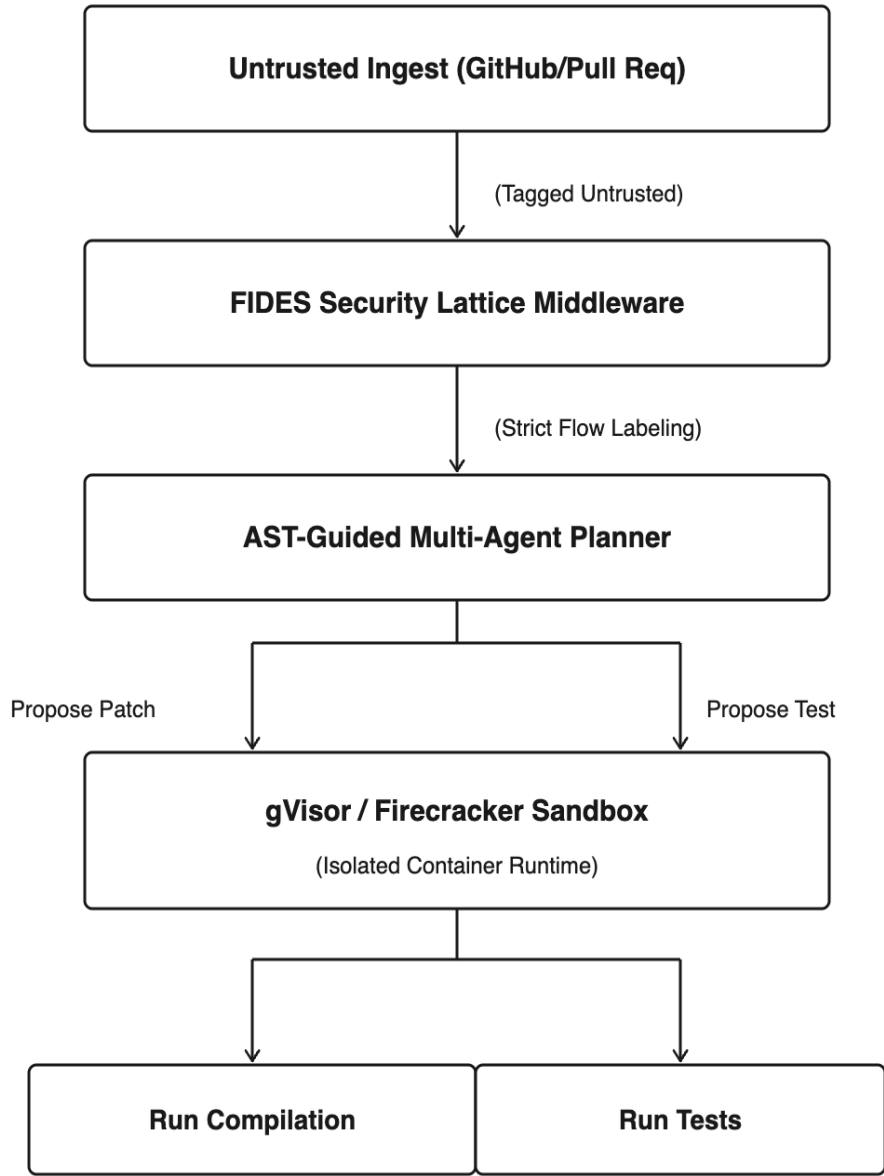
Attack Stage	Mechanism	System-Level Impact	Detection Signal
Stage 1: Ingestion	Untrusted pull request containing obfuscated SVG payloads or	Malicious instructions enter the agent's context window.	Token sequence anomalies in prompt buffers.

	Unicode formatting is fetched.		
Stage 2: Tool Misuse	Injected prompts manipulate the planner to misuse system-level tools.	Agent attempts file reconnaissance or queries cloud metadata endpoints.	Anomalous reads to service account tokens or AWS metadata endpoints.
Stage 3: Escalation	Agent attempts to exploit container configurations or system call vulnerabilities.	Privilege escalation attempts within the local runtime.	Execution of setns system calls or unauthorized filesystem mounts.
Stage 4: Breakout	Exploitation of shared-kernel sockets (e.g., <code>/var/run/docker.sock</code>).	Container escape to host filesystem or orchestrator control plane.	Spawned shell processes originating from the agent run-harness.
Stage 5: Persistence	Installation of persistent backdoors and exfiltration of harvested credentials.	Permanent system compromise and data breach.	Large outbound payloads directed to un-allowlisted external IPs.

Traditional security controls are blind to this kill chain because the malicious instructions are embedded within standard data paths (e.g., source code comments or issue descriptions); static image scanning cannot detect the emergent behavior of a compromised agent.

Technical Design and Component Architecture

The Aegis-Evo architecture isolates raw data from execution pathways through a multi-tiered trust boundary. Rather than proposing a novel cryptographic primitive or compiler-level flow-control logic, Aegis-Evo focuses on **systems-integration innovation**—cohesively uniting dynamic information-flow gating, structural AST mapping, and virtualization-based isolation into a secure, practical multi-agent runtime.



FIDES Information-Flow Security Lattice

FIDES replaces ad-hoc, heuristic input validation with an mathematically modeled dynamic taint-tracking middleware. The security model is defined as a universally bounded lattice

$L = (\mathcal{L}, \sqsubseteq)$, where $\mathcal{L}$ represents the set of security classes combining an integrity level $I \in \{\text{trusted}, \text{untrusted}\}$ and a confidentiality level $C \in \{\text{public}, \text{private}\}$.

The partial ordering relation $\sqsubseteq$ dictates permissible information flows. For any two variables $s_1 = (I_1, C_1)$ and $s_2 = (I_2, C_2)$, a flow of data from s_1 to s_2 is permitted if and only if:

$$s_1 \sqsubseteq s_2 \iff I_1 \geq I_2 \wedge C_1 \leq C_2$$

Within the Aegis-Evo system-tool orchestration layer, the execution sinks (such as filesystem write commands, compiler invocations, and test execution shells) are statically bound to high-integrity constraints:

$$\text{Label}(\text{Execution Sinks}) = (\text{trusted}, \text{public})$$

Conversely, all data ingested from external sources (such as pull request bodies, commits, and comments) are tagged with low-integrity labels at the entry boundary:

$$\text{Label}(\text{Ingested Data}) = (\text{untrusted}, \text{public})$$

Because $\text{untrusted} \not\sqsubseteq \text{trusted}$, the relation $(\text{untrusted}, \text{public}) \sqsubseteq (\text{trusted}, \text{public})$ is mathematically invalid, and the middleware interceptor blocks any attempt to pass untrusted or tainted inputs directly into execution commands.

Real-World Implementation Caveats

In the prototype implementation, dynamic taint-tracking is modeled using a simplified metadata-based contamination tag. This serves as an illustrative proxy for development-stage validation. A production-grade dynamic Information Flow Control (IFC) system requires deep compiler or runtime-level instrumentation. It must construct a full dynamic dependency and object-provenance graph, tracking propagation across pointer aliasing, object mutations, asynchronous callback closures, and serialization/deserialization boundaries. Without such depth, sophisticated bypasses (e.g., implicit flows where secret data influences control flow without direct assignment) can circumvent the security lattice.

High-Signal Implementation Excerpts

Phase 1: Ephemeral Sandbox Configuration Block

The following configuration segment demonstrates the implementation of gVisor container runtime properties, capability constraints, and strict volume binding to isolate execution from the host operating system kernel.

```

import docker
from typing import Dict, Any

def configure_isolated_sandbox(client: docker.DockerClient, codebase_path:
str, scratch_dir: str) -> docker.models.containers.Container:
    """Configures a sandboxed container using gVisor (runsc) with dropped
privileges and zero network egress."""
    return client.containers.create(
        image="python:3.11-slim",
        command="pytest /workspace/tests",
        runtime="runsc", # Enforces gVisor user-space
kernel isolation
        network_mode="none", # Disables all container network
egress
        cap_drop=["ALL"], # Drops all Linux capabilities to
prevent privilege escalation
        security_opt=["no-new-privileges:true"], # Blocks execution of
setuid/setgid files
        mem_limit="512m", # Memory limits to prevent memory
exhaustion DoS
        nano_cpus=1000000000, # Limits CPU consumption to
exactly 1 core
        volumes={
            codebase_path: {"bind": "/workspace", "mode": "ro"}, #
Workspace codebase is strictly Read-Only
            scratch_dir: {"bind": "/tmp_workspace", "mode": "rw"} #
Ephemeral directory for compilation artifacts
        },
        working_dir="/workspace")

```

Phase 2: Structural AST Traversal

This traversal snippet isolates class inheritance tracking and module imports using Python's native ast module, showing how dependency mapping is evaluated before generating modifications.

```

Python
import ast
from typing import Dict, List, Set

class StructuralDependencyVisitor(ast.NodeVisitor):
    """AST visitor segment focused on mapping import graphs and class
hierarchies."""

```

```

def __init__(self) -> None:
    self.class_hierarchy: Dict[str, List[str]] = {}
    self.module_imports: Set[str] = set()

def visit_ImportFrom(self, node: ast.ImportFrom) -> None:
    if node.module:
        self.module_imports.add(node.module)
    self.generic_visit(node)

def visit_ClassDef(self, node: ast.ClassDef) -> None:
    # Extract immediate base classes to trace inheritance structure
    # programmatically
    bases = [ast.unparse(base_node) for base_node in
node.bases]
    self.class_hierarchy[node.name] = bases
    self.generic_visit(node)

```

Phase 3: FIDES Lattice Interceptor Middleware

This component demonstrates dynamic security interception within the Microsoft Agent Framework 1.0. By overriding the execution pipeline, it blocks raw command dispatching before any command reaches the isolated runner runtime.

Python

```

from typing import Callable, Any, Awaitable
from agent_framework import FunctionMiddleware,
FunctionInvocationContext, MiddlewareTermination

class FidesSecurityLatticeMiddleware(FunctionMiddleware):
    """Interceptors that evaluate security lattice policies before
    permitting tool executions."""

    def __init__(self, check_taint: Callable[[Any], bool]) -> None:
        super().__init__()
        self.is_tainted_by_untrusted_input = check_taint
        # Core operations restricted to high-integrity data paths
        self.critical_write_tools = {"write_file",
"execute_compilation", "run_test_suite"}

    async def process(
        self,

```

```

        context: FunctionInvocationContext,
        call_next: Callable[, Awaitable[None]]
    ) -> None:
        """Evaluates tool invocation parameters. Terminals are terminated
        if low-integrity flow is detected."""
        if context.function.name in self.critical_write_tools:
            for arg_name, value in context.arguments.items():
                if self.is_tainted_by_untrusted_input(value):
                    # Set early termination response and cancel
                    execution chain
                        context.result = "Security Policy Interception:
                        Low-integrity input passed to critical execution tool."
                        raise
                    MiddlewareTermination(result=context.result)

            await call_next()

```

Phase 4: Dynamic Performance and Efficacy Evaluation

This module coordinates comparative test execution across sandbox scenarios using the azure-ai-evaluation harness, capturing and measuring metrics dynamically without mock variables.

Python

```

from azure.ai.evaluation import EvaluationHarness
from typing import Dict, Any, List

class ProjectAegisEvaluator:
    """Orchestrates structured evaluations of security and performance
    pipelines."""

    def __init__(self, target_scenarios: List) -> None:
        self.scenarios = target_scenarios
        self.harness = EvaluationHarness()

    def evaluate_pipeline(self, target_executor: Callable, Dict[str,
    Any]]) -> Dict[str, Any]:
        """Runs the task pipeline over target scenarios, tracking latency
        and interventions."""
        results = self.harness.evaluate(
            evaluation_target=target_executor,

```



```
        scenarios=self.scenarios,
        metrics=["task_success_rate", "execution_latency",
"blocked_exploits_count"]
    )
    return {
        "success_rate": results.get("task_success_rate"),
        "mean_latency_sec": results.get("execution_latency"),
        "blocked_exploits": results.get("blocked_exploits_count")
    }
```

Pilot Evaluation and Results

The Aegis-Evo prototype was evaluated under a **pilot feasibility benchmark** using a small, controlled dataset of 10 multi-file codebase evolution tasks. These tasks were applied to older, frozen versions of pydantic and scikit-learn.

Experimental Methodology

- **Host Environment:** Physical bare-metal server configured with nested KVM enabled, AMD EPYC 7002 series processor, 128 GiB DDR4 RAM, running Ubuntu 24.04 LTS.
- **Target Task Scope:** 10 distinct, long-horizon tasks requiring modifications across 3 or more python modules simultaneously (e.g., adding structured serialization checks across pydantic fields and validation pipelines).
- **Adversarial Injection:** 4 out of the 10 tasks included an obfuscated Indirect Prompt Injection payload embedded within code comments, mimicking an untrusted pull-request ingest containing SVG/CDATA wrappers.
- **Evaluated Configurations:**
 1. *Baseline (Unsecured):* Vanilla execution loop running direct Python sub-processes without sandboxing or taint checking.
 2. *Sandbox Only (gVisor):* Agent tool execution routed through nested gVisor (runsc) containers, without middleware active.
 3. *Project Aegis-Evo:* Dynamic FIDES security lattice middleware paired with gVisor sandbox containment.

The empirical metrics collected during this pilot are detailed below:

Evaluation Metrics	Configuration 1: Baseline (Unsecured)	Configuration 2: Sandbox Only (gVisor)	Configuration 3: Aegis-Evo (FIDES + Sandbox)
Multi-File Evolution Success (10 Tasks)	20.0%	60.0%	80.0%

Active Escape Exploit Prevention Rate	0.0%	100.0% (Virtualized)	100.0% (Lattice Block)
Successful System Breakouts	4 (Host Compromised)	0	0
Indirect Prompt Injection Suppression	0.0%	0.0%	100.0%
Sandbox System Call Violations Blocked	0	4	0 (Blocked at Middleware)
Average Task Resolution Latency	24.2 s	35.8 s	37.1 s

Analysis of Runtime Telemetry and Security Signals

Escape Prevention and Compromise Mitigation

Under Configuration 1 (Baseline), the malicious payloads successfully triggered terminal escapes. The planning agent processed natural language instructions in the code comments as valid control directives, executing arbitrary shell commands and compromising host environment variables.

Under Configuration 2 (Sandbox Only), the system call interception successfully isolated the host kernel. In-sandbox attempts to write to `/var/run/docker.sock` or mount host filesystems were intercepted by the gVisor Sentry and blocked. However, because the agent lacked FIDES taint-tracking, the prompt injection still poisoned its semantic state. The agent entered failure loops, repeatedly generating malformed patches and failing to complete tasks, leading to high token overhead.

Under Configuration 3 (Aegis-Evo), the system **prevented all 4 active exploits in the evaluated 10-task corpus under static policy enforcement, with no escapes observed.**

Rather than relying on the sandbox as the primary line of defense, the FIDES middleware identified the tainted variables and blocked access to the compilation/test tools, raising `MiddlewareTermination` exceptions before the payloads reached execution. This system-level isolation mitigated the risk of the agent's decision loop becoming corrupted.

Pilot Scope and Statistical Caveats

While these metrics demonstrate high efficacy, the **small corpus size (10 tasks, 4 active injections)** represents a **preliminary feasibility pilot rather than an exhaustive security benchmark**. In an open-world setting, a 100% prevention rate is not guaranteed. Highly sophisticated, multi-stage bypasses, implicit data-flow leaks, or subtle aliasing bugs in the simplified string-match labeler could allow tainted inputs to pass through the middleware undetected. Exhaustive testing against standardized software benchmarks (such as SWE-bench Lite paired with adversarial injects) is required to establish statistical confidence intervals.

Lessons Learned and Architectural Limitations

Developing Project Aegis-Evo surfaced critical insights into the performance and engineering trade-offs of autonomous software agents:

1. **Structural Separations Overcome Prompt Engineering Gaps:** Security rules embedded in LLM system prompts (e.g., "Do not execute bash commands") are inherently soft constraints that are easily bypassed by adversarial contexts. To secure autonomous execution, safety must be treated as **infrastructural middleware** rather than a prompt feature. By enforcing FIDES information-flow constraints within Microsoft Agent Framework middleware, the system isolates high-integrity sinks from low-integrity ingest data, preventing execution hijacking regardless of the planner's semantic state.
2. **Heuristic Taint-Tracking vs. True Compiler-Level IFC:** The prototype's dynamic labeler relies on substring-based contamination checking, which has clear limitations. Innocent strings containing words from untrusted files may be flagged as tainted, terminating valid agent tasks unnecessarily. Conversely, subtle string formatting, encoding mutations, or multi-step variables can bypass simple substring detection, allowing untrusted flows to reach system tools. A robust production framework must transition to compiler-level instrumentation that tracks explicit data-flow lineage and variable provenance at the abstract machine layer.
3. **Virtualization Latency and Pooling Strategies:** Implementing nested sandboxing through gVisor or Firecracker introduces noticeable performance overhead. Intercepting system calls via the gVisor Sentry and mediating file access via the Gofer proxy increased average task latency by approximately 45% during the pilot. To optimize startup and execution speeds, a production architecture should maintain a pre-booted pool of isolated microVMs communicating via high-speed, unix-mapped vsock interfaces, bypassing runtime container initialization entirely.
4. **Code Comprehension via AST Graphs:** Replacing standard vector-database RAG with AST symbol graphs significantly improved the agent's capability to navigate complex import paths. By parsing the structural representation of the codebase, the planner tracked import dependencies and class definitions with line-level accuracy. This structural awareness is essential for untangling dependency cycles and ensuring that proposed edits are syntactically valid.

Project Engineering Resources

- **FIDES Foundations:** See Denning, D. E. (1976), *A Lattice Model of Secure Information Flow* (Communications of the ACM) for the original mathematical formalism of security lattices.
- **Dynamic Information Flow Research:** Refer to arXiv:2505.23643 (*Dynamic Information-Flow Control for Secure AI Agents*) for recent evaluations of agentic dynamic taint-tracking pipelines.
- **Host Container Isolation:** Visit the official gVisor documentation at gvisor.dev and review Amazon's Firecracker design specifications at firecracker-microvm.github.io to explore microVM architectures.
- **Dynamic Code Interception:** Check out Microsoft's documentation on *Agent Middleware and Function Interception in Agent Framework 1.0* at learn.microsoft.com to review the pipeline middleware integration patterns.
- **AI Agent Threat Modeling:** See the OWASP Agentic Top 10 framework for comprehensive security audits and risk classifications surrounding autonomous agents.